De la Teoría al Código: Implementando *K-Armed Bandits* con Patrones de Diseño.

Autor: Marco Aurélio Guimarães Falcão – Ingeniero de Sistemas

Fecha: 10/08/2026 **Version:** Español, Traducida del Portugués por la IA Claude

1. Introducción

Este artículo trata de la implementación en Python de algoritmos de solución para un problema clásico del Aprendizaje por Refuerzo, el problema de los K-bandidos descrito en el Capítulo 2 del libro de [Sutton & Barto] [1]. Este problema introduce los conceptos más básicos del Aprendizaje por Refuerzo, como por ejemplo: Entorno, Agente, Acciones, Políticas y Recompensas. El libro [1] también presenta las principales estrategias de solución del problema de los K-bandidos: Método de Estimación del Valor de la Acción, Valor Inicial Optimista, Límite Superior de Confianza (UCB – *Upper Confidence Bound*), y Método de los Gradientes.

El Capítulo 1 del libro [Lattimore & Szepesvári] aborda el concepto de Arrepentimiento como una métrica de desempeño del aprendizaje del agente muy utilizada en la comparación del uso de un mismo algoritmo en diferentes entornos. La sección 36.1 del mismo libro aborda la estrategia de solución por *Thompson-Sampling* (TS), una alternativa de mejor desempeño pero con mayor costo computacional.

Para la implementación de las estrategias de solución mencionadas anteriormente se utilizaron los patrones de diseño ***Strategy*** y ***Template Method***. El primero posibilita la fácil reutilización de estas soluciones en diversos casos de prueba, mientras que el segundo permite la abstracción y el uso de una interfaz estandarizada, optimizando la implementación de la jerarquía de clases de solución.

Se utilizaron algunos ejemplos del Capítulo 2 del libro de [Sutton & Barto] [1] como casos de prueba para validar la implementación en Python, en los cuales se obtuvieron resultados similares para entornos estacionarios. Además, se exploraron casos de prueba para entornos no estacionarios, como en el ejercicio 2.11 de ese libro.

2. Presentación General de RL (*Reinforcement Learning*) y Definición del Problema.

La Figura 1 presenta el Esquema General del Aprendizaje por Refuerzo, ilustrando la interacción cíclica entre un Agente y un Entorno. El Agente ejecuta acciones que impactan el Entorno; este, a su vez, reacciona proporcionando observaciones —que pueden indicar su estado interno de forma directa o indirecta— y recompensas, que sirven como métricas de éxito o fracaso de las acciones tomadas. El Agente procesa ese *feedback* a lo largo del

tiempo, buscando estimar el valor esperado (o valor medio) de la recompensa para cada acción. Con base en esas estimaciones y en el estado actual del Entorno, el Agente determina la siguiente acción a tomar, siguiendo una estrategia o política definida.

El Arrepentimiento es la pérdida acumulada de recompensa sufrida por el agente a lo largo del tiempo, resultante de elegir acciones subóptimas en lugar de la acción de mayor valor esperado (acción óptima).

De forma intuitiva, el Aprendizaje por Refuerzo puede compararse con el proceso de educar a un cachorro al llegar a un nuevo hogar. Inicialmente, movido por la exploración, el animal realiza diversas acciones aleatorias: roe la pata de la silla, destruye calcetines o hace sus necesidades en lugares inadecuados. El aprendizaje ocurre a través de la intervención del tutor, que actúa como mediador de las recompensas. Ante cada comportamiento deseado, el cachorro recibe un refuerzo positivo (como una galleta); ante cada acción inconveniente, recibe un refuerzo negativo (como una llamada de atención). Con el tiempo y la repetición, el pequeño animal integra ese historial de *feedback* y aprende a seleccionar las acciones que maximizan sus recompensas, comprendiendo las 'reglas' del entorno de la casa.

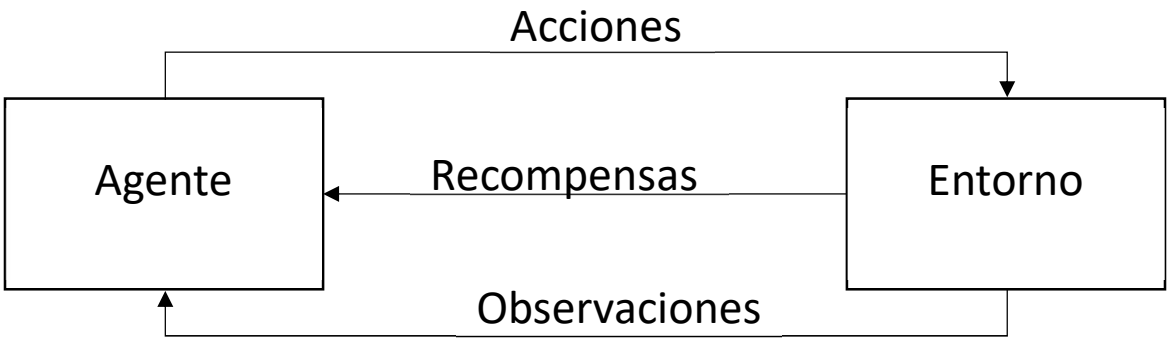


Figura 1 – Esquema General del Aprendizaje por Refuerzo

La estrategia de solución del problema del Aprendizaje por Refuerzo aborda «Cómo» elegir la mejor acción y se conoce como Política. Una solución ***Greedy* (Voraz)** busca siempre elegir la acción con mayor recompensa media. Esta técnica se llama **explotación**. Para evitar que el Agente quede 'atrapado' en una elección apenas razonable, es fundamental permitir la **exploración**: la práctica de experimentar, ocasionalmente, otras acciones con medias inferiores o desconocidas. Un ejemplo cotidiano de este dilema es el hábito de frecuentar siempre el mismo restaurante por ya conocer su relación costo-beneficio (explotación) frente a la decisión de, ocasionalmente, visitar un lugar nuevo para descubrir sabores superiores (exploración).

El Arrepentimiento es la métrica ideal para evaluar algoritmos en simulación porque captura el costo de la exploración de forma más precisa que la recompensa bruta. Sin embargo, su cálculo es inviable en el mundo real, en el cual no conocemos la recompensa de la acción óptima. Como en la vida, el arrepentimiento es ese sentimiento de probar un restaurante nuevo y darse cuenta de que debería haber ido al de siempre.

Para la comprensión inicial del problema, se recurre a la etimología de *'One-Armed Bandit'* [3] (el 'Bandido de un Solo Brazo'). El término deriva de la unión de *one-armed* (que se refiere a la palanca única que opera las máquinas tragamonedas clásicas) y *bandit* (un ladrón que actúa en zonas sin ley o alguien que engaña y perjudica a otros). Esta metáfora refleja el hecho de que estas máquinas están diseñadas para 'robar' el capital de los jugadores que, estadísticamente, pierden frente a la banca a lo largo del tiempo.

El problema de los **K-Bandidos** es una extensión matemática del dilema del *'One-Armed Bandit'* para un escenario con K máquinas tragamonedas. En este modelo, un operador (o **Agente**) busca identificar y seleccionar la palanca de la máquina que proporciona el mayor retorno medio de recompensas a lo largo del tiempo, como se ilustra en la Figura 2. El desafío central reside en que las distribuciones de probabilidad de recompensas de cada máquina son inicialmente desconocidas, lo que exige que el Agente equilibre la exploración de nuevas máquinas con la explotación de aquellas que ya han demostrado buenos resultados. El problema de los K bandidos también se conoce en la literatura como MAB (*Multi-Armed Bandits Problem*).

Al describir el problema del Aprendizaje por Refuerzo, es fundamental distinguir los dos tipos de entornos considerados: el **Estacionario** y el **No Estacionario**. En el entorno estacionario, las propiedades estadísticas de las recompensas (como la media y la desviación estándar de las recompensas de cada brazo) permanecen constantes a lo largo del tiempo. En el entorno no estacionario, en cambio, esas estadísticas cambian con el tiempo, lo que exige que el agente sea capaz de adaptar sus estimaciones continuamente, modificando y actualizando la Política de forma dinámica. Esta distinción es crucial, pues determina la elección de la estrategia de actualización y el peso dado a las recompensas más recientes.

Una Política π define cómo debe elegirse una acción $a \in A$ en función de las observaciones obtenidas que definen el estado s del Entorno en cuestión, como se muestra en la Ecuación (1). Es decir, la Política se traduce en el conjunto de probabilidades de elección de la acción a perteneciente al conjunto de acciones posibles *A* dado que el entorno se encuentra en el estado s.

$$\pi = \text{Prob}(a \in A/s) \quad (1)$$



Figura 2 – El Problema de los K Bandidos – Generada por Gemini 3 Flash Image

El problema de los K-Bandidos se considera la puerta de entrada más accesible al Aprendizaje por Refuerzo por tratarse de un modelo de estado único. A diferencia de problemas más complejos, aquí el entorno no cambia de configuración (no cambia de estado) tras una acción; el foco reside en la estimación de las recompensas y en la optimización de la política de elección. Entornos más sofisticados exigen el tratamiento de cambios de estado probabilísticos, como los Procesos de Decisión de Markov (MDP), o la consideración de restricciones de costo o esfuerzo para realizar la acción, lo cual escapa al alcance de este texto.

Supongamos que la máquina 3 de las 10 máquinas de la Figura 2 posee la mejor recompensa media μ^* respecto a todas las máquinas. La política óptima π^* se definiría por el conjunto de probabilidades mostrado en (2). Es decir, jugando siempre en la máquina 3 se obtiene la mejor recompensa media. El agente desconoce cuál es la mejor máquina, por lo que busca aprender a partir de las recompensas cuál es la mejor estrategia para elegir la acción, en este caso, la máquina a elegir.

$$\pi * = [0, 0, 1, 0, 0, 0, 0, 0, 0, 0] \quad (2)$$

El problema de los K-bandidos tiene varias aplicaciones prácticas destacables, entre ellas [2][4]:

- Recomendación de Contenido – presentación de artes personalizadas de las películas para cada usuario por parte de Netflix. Las mejores artes captan la atención de los clientes para ver películas o series [5];
- Pruebas A/B adaptativas – elección del mejor arreglo de componentes gráficos (*Widgets*) en una página Web que resulte en mejores resultados, es decir, más compras realizadas por el cliente;
- Ensayos Clínicos - dosificación de medicamentos con menor costo ético. Permite a los profesionales de la Medicina experimentar tratamientos en pacientes con medicamentos observando sus resultados;
- Redes de Drones – optimización de trayectorias y asignación de recursos (potencia y batería) en misiones de rescate. En un área de desastre, la infraestructura de la red celular queda destruida y un dron puede utilizarse como ERB (Estación Radio Base) para establecer comunicación con las víctimas. En este caso, los bandidos son un número discreto de direcciones posibles que el dron puede asumir; para cada dirección se miden Recompensas en términos de mejor comunicación con las víctimas, sujetas a restricciones de costo de batería y potencia de transmisión. En función de estas restricciones, el número de direcciones posibles se modifica de forma dinámica [6]; y
- Finanzas – Selección dinámica de carteras y estrategias de *trading* de acciones que se adaptan rápidamente a cambios de régimen en el mercado. En una estrategia de inversión mensual de nuevos aportes o dividendos, elegir en qué activo de la cartera invertir.

Este estudio fue desarrollado mediante un proceso colaborativo con diversas Inteligencias Artificiales (IAs) utilizando recursos libres, abarcando desde la lluvia de ideas conceptual y la estructuración del artículo hasta la elaboración, revisión y corrección de código, la definición de la arquitectura de clases, la generación de imágenes y la indicación de referencias. Se optó deliberadamente por no utilizar automatizaciones de agentes para la generación integral y automática del texto. En cambio, el autor orquestó la construcción del contenido a través de diálogos iterativos con diferentes modelos, priorizando dos objetivos fundamentales: la profundización técnica en el tema del Aprendizaje por Refuerzo y la exploración de las capacidades y matices de las propias IAs utilizadas. Para este estudio se emplearon dos enfoques: construcción incremental (*Build-a-little Test-a-Little*) y *Human-in-the-Loop* para la elaboración de preguntas y pruebas/validación de las respuestas de las IAs. Las herramientas específicas de IA empleadas están debidamente citadas en la Conclusión.

El lenguaje Python se utiliza para la implementación de las soluciones. Por ser el lenguaje predominante en el área de IA, dispone de bibliotecas que facilitan la codificación, como el paquete ***Gymnasium*** (o Gym) [7], que estandariza la interfaz entre el agente y el entorno. De manera complementaria, el uso de **Google Colab** posibilita el desarrollo del código de forma progresiva y colaborativa, ofreciendo soporte de GPU e integración con la IA Gemini, lo que garantiza mayor agilidad en la ejecución de las pruebas y experimentos realizados.

3. **Estrategias de Solución:**

Las estrategias de solución se basan en la estimación del valor esperado de las recompensas para cada acción (palanca), conforme definido por la Media Muestral (3).

Sea R_i la recompensa obtenida tras el i-ésimo accionamiento de una determinada palanca: el valor estimado de la acción después de haber sido seleccionada n-1 veces, denotado por Q_n, se calcula mediante la media aritmética de las recompensas recibidas, con i variando de 1 a n-1

$$Q_n = \frac{R_1 + R_2 + \ldots + R_{n-1}}{n-1} \tag{3}$$

Según la Ley de los Grandes Números, a medida que n aumenta, la estimación Qn converge al valor real de la recompensa media de la palanca q*.

La representación recursiva presentada en la Ecuación (4) es matemáticamente equivalente a la media aritmética, pero más eficiente para la implementación computacional. Aísla el **error de estimación** —la diferencia entre la recompensa actual y la media anterior— y lo escala por un factor de paso (1/n), permitiendo que la estimación converja de forma incremental hacia el valor real q*.

$$Q_{n+1} = Q_n + \frac{1}{n}[R_n - Q_n] \tag{4}$$

La Ecuación (4) puede interpretarse como una regla de actualización basada en el error de estimación, donde el nuevo valor se ajusta a partir de la media anterior sumada a un incremento ponderado. Este incremento, representado por el término (R_n - Q_n), se denomina técnicamente **error de predicción** o **innovación**.

La estructura general de esta actualización se expresa en la Ecuación (5), en la cual el peso variable 1/n se sustituye por un parámetro de tamaño de paso (o tasa de aprendizaje) constante α.

$$Q_{n+1} = Q_n + \alpha[R_n - Q_n] \quad (5)$$

Una vez calculadas las estimaciones mediante las Ecuaciones (4) o (5), el agente debe seleccionar la próxima acción a ejecutar. El criterio más común es el método ***Greedy* Puro (Voraz)**, en el cual el agente selecciona la opción que presenta la mayor estimación de Valor de la Acción en el momento actual. Formalmente, la acción seleccionada en el paso t, denotada por A_t, se define como:

$$A_t = argmax_{(a \in A)}[Q_t(a)] \quad (6)$$

Sin embargo, las estimaciones de valor de la acción son variables estadísticas que presentan desviaciones respecto a la media real q*. Por el **Teorema del Límite Central**, el error de esas estimaciones tiende a una distribución normal con media 0 y desviación estándar dada por
$\frac{\sigma}{\sqrt{(n)}}$. En consecuencia, la elección de la acción con la mejor estimación actual no garantiza el mejor desempeño a largo plazo, dado que el valor de $Q_t(a)$ puede estar momentáneamente distorsionado por fluctuaciones muestrales.

Para mitigar este riesgo, se utiliza la estrategia **ε-Greedy**. En este método, el agente alterna entre:

- **Explotación (*Exploitation*):** Con probabilidad 1-ε, se selecciona la acción de mayor valor estimado.
- **Exploración (*Exploration*):** Con probabilidad ε, se selecciona una acción aleatoria dentro del espacio $\mathcal{A}$ de acciones, permitiendo al agente refinar sus estimaciones sobre acciones potencialmente subestimadas.

Cabe destacar que el Valor de Acción para cada bandido debe partir de un valor inicial en el instante 0, Q_0. Es común inicializar Q_0 en 0 para todos los K bandidos. Sin embargo, existe una solución que inicializa Q_0 con valores más altos e iguales para todos los K bandidos. Dicha solución se conoce como **Valor Inicial Optimista** (*Optimistic Initial Value*) y utiliza la regla establecida por la Ecuación (6) para explorar todos los K bandidos justo al inicio del algoritmo.

Esto permite obtener un mejor desempeño mediante una exploración sistemática y completa de todas las acciones, lo que generalmente acelera la identificación de la acción óptima en problemas **estacionarios**, en relación con el método ***Greedy* Puro**. Sin embargo, dado que el método **ε-Greedy** ya posee una exploración implícita, **no se recomienda** usar el Valor Inicial Optimista con él, pues esta combinación puede saturar la exploración e incluso empeorar el desempeño de la solución.

En entornos no estacionarios, esta técnica pierde eficiencia, ya que la exploración inicial resulta insuficiente para seguir el ritmo de los cambios del sistema a lo largo del tiempo. En estos escenarios se observa una mejora al adoptar un tamaño de paso constante más elevado, lo que permite al algoritmo priorizar las recompensas recientes en detrimento de las estimaciones históricas. Sin embargo, los experimentos realizados en este estudio revelan que esta ganancia ocurre solo en los pasos iniciales, como se detalla en la sección de resultados.

En el presente artículo y en el capítulo 2 del libro Sutton & Barto [1], las recompensas del entorno estacionario para cada Acción en el instante n, Rn, se generan a partir de distribuciones normales con varianza unitaria, cuyas medias son, a su vez, sorteadas de una distribución normal estándar - $\mathcal{N}$ ($\mathcal{N}$(0,1), 1), es decir, las medias de recompensa para cada bandido μ1, μ2,..., μk tienen media cero y desviación estándar unitaria, y la recompensa en sí tiene media μi y desviación estándar unitaria para cada bandido i $\in$ [1, K]. Los valores esperados (medias reales) de recompensa para cada brazo, μ1, μ2,.., μk, se muestrean inicialmente de una distribución normal estándar $\mathcal{N}$ (0, 1) y ya no cambian más, de modo que el máximo de μi, μ* según la Ecuación (7), ya indica el bandido que produce mejores recompensas medias, lo que resulta útil para evaluar el porcentaje de elecciones óptimas del algoritmo.

$$\mu_* = max_{(a \in A)}[\mu_1, \mu_2, \ldots, \mu_k] \tag{7}$$

Para el entorno No Estacionario, las medias de las Normales utilizadas para el sorteo y generación de las recompensas se modifican en cada paso mediante la suma de una desviación obtenida de una distribución Normal *N*(0, 1) como en la Ecuación (8).

$$[\mu_1, \mu_2, \ldots, \mu_k] += N(0,1) \tag{8}$$

Para el cálculo del porcentaje de elección óptima, se adopta como referencia la palanca de mayor media inicial, fundamentada en la expectativa teórica del sistema. Dado que las perturbaciones añadidas en cada paso provienen de distribuciones normales de media nula, la esperanza matemática a largo plazo de las recompensas permanece inalterada, conservando la identidad de la palanca óptima original. De esta forma, la métrica mide la robustez del algoritmo para converger hacia la mejor elección teórica incluso bajo la acumulación de fluctuaciones estocásticas a lo largo del tiempo.

Vimos cómo funciona el método **ε-Greedy**, que puede considerarse análogo a una **cola de prioridades**. En ella, la prioridad se basa en la estimación de Valor de la Acción (Q) para cada palanca, recalculada en cada paso del algoritmo para el bandido seleccionado. Eventualmente, un "juez" realiza un sorteo entre todos los brazos para decidir quién debe pasar al frente de la cola, basándose en una probabilidad fija ε.
De esta forma, la regla de exploración se fundamenta en el par **(Q, ε)** , siendo Q la Estimación del Valor de la Acción para cada brazo y ε un valor fijo generalmente pequeño. La gran limitación de este enfoque es que una palanca prometedora puede, por simple mala suerte, tardar mucho en ser seleccionada nuevamente, haciendo que la exploración sea poco efectiva y esté sujeta al olvido de acciones potencialmente óptimas.
Continuando con la analogía, el método **UCB (*Upper Confidence Bound*)** busca usar información adicional para definir la prioridad de la cola, como el tiempo transcurrido t –equivalente al número de pasos del algoritmo– y el número de veces que cada palanca es accionada N. De esta forma, la regla de prioridad se establece con base en la tupla **(Q, t, N).** El Teorema del Límite Central establece que la desviación estándar respecto a la estimación de la media es igual a $\frac{\sigma}{\sqrt{N}}$ y el intervalo de confianza en torno a la estimación de la media corresponde a $\pm c.\frac{\sigma}{\sqrt{N}}$ donde c es una constante. El límite superior del intervalo de confianza es igual a $Q + c.\ \frac{\sigma}{\sqrt{N}}$. Partiendo de los Fundamentos en Límites de Probabilidades - Desigualdad de Hoeffding, el método UCB establece mediante la ecuación (9) un límite superior de intervalo de confianza para las estimaciones de las medias de valor de las acciones de cada palanca, de forma que aumenten las posibilidades de que sea seleccionada cuanto mayor sea el tiempo transcurrido t y cuanto menor sea el número de veces que ha sido seleccionada en el pasado Nt(a). Así, logra una mejor exploración de las palancas, sorteando el problema del posible olvido de una palanca en el método **ε-Greedy.**

$$A_t = argmax_{(a\in A)}[Q_t(a)+c.\sqrt{\frac{log(t)}{N_t(a)}}] \quad (9)$$

Obs.: c = 1 o c = 2 son valores típicamente usados con la ecuación (9).

La Figura 3 presenta el Diagrama de Bloques del Método del Gradiente. Este método se basa en la búsqueda de una política óptima π, que se inicia como una distribución uniforme (donde todos los K bandidos son equiprobables), siendo actualizada en cada ciclo con base en un índice conocido como H (Preferencia de la Acción). Esta actualización de las preferencias puede orientarse por la desviación de la Recompensa Actual respecto a la media de las recompensas pasadas (con Baseline) o basarse puramente en el valor bruto de la Recompensa Actual (sin Baseline). Por último, el vector H se mapea mediante la función Softmax (H) para generar la nueva distribución de probabilidades de la política π para el siguiente ciclo.

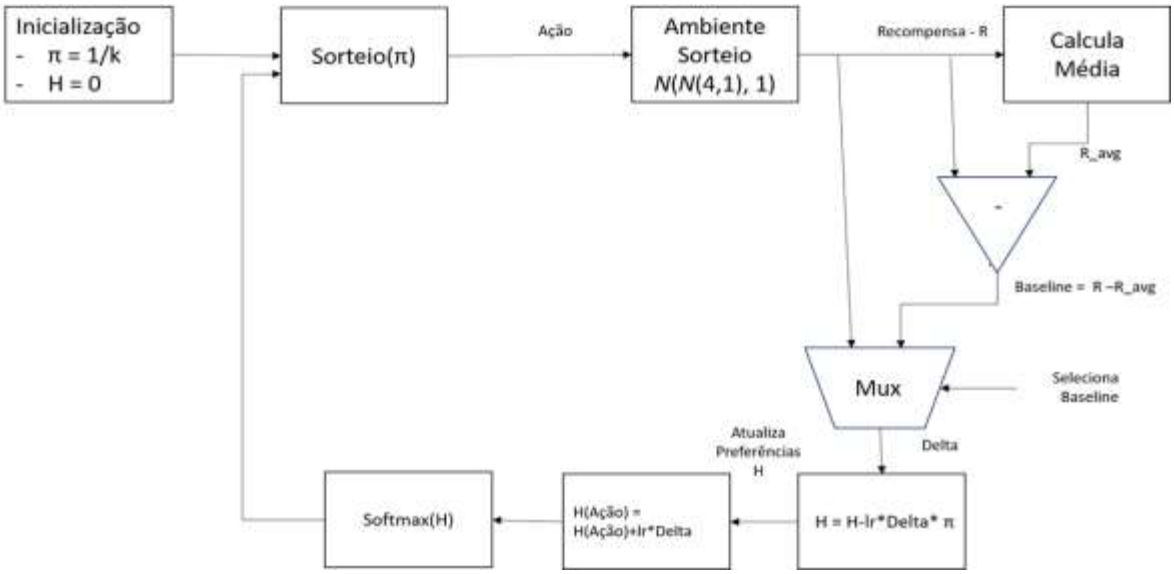


Figura 3-Diagrama de Bloques del Método del Gradiente

Obs.: El hiperparámetro del Método del Gradiente es la tasa de aprendizaje lr (*Learning Rate*).

La sección 36.1 de la página 461 de la referencia [2] presenta la estrategia de solución del problema de los K bandidos armados mediante *Thompson-Sampling* (TS). El TS es una estrategia probabilística para el problema de los K bandidos que estima, para cada acción, los parámetros de media y varianza (o desviación estándar) de la distribución de recompensas del entorno. Inicialmente, se asume un conocimiento a priori, asignando los

mismos valores iniciales de media y desviación estándar a todas las acciones. En cada paso, se muestrean valores de la distribución a priori para cada acción y se selecciona la que maximiza el valor muestreado. Al accionar la palanca elegida, se observa una recompensa, que luego se utiliza para actualizar los parámetros de la distribución a posteriori mediante el Teorema de Bayes. Esta distribución actualizada se convierte en el nuevo prior para el paso siguiente, y el proceso se repite a lo largo del horizonte temporal. Con esto, el TS converge progresivamente hacia una buena estimación de las distribuciones reales de recompensa del entorno.

El agente *Thompson Sampling* (TS) empleado en este artículo se basa en el uso de distribuciones normales para modelar la creencia sobre la recompensa media de cada acción. Este enfoque resultó adecuado para los escenarios investigados, en los cuales el entorno también genera recompensas a partir de distribuciones normales. Aunque el TS con prior normal funciona satisfactoriamente en una amplia gama de problemas —gracias a la robustez de la actualización Bayesiana y al Teorema Central del Límite— existen situaciones en las que otras distribuciones son más apropiadas. Ejemplos incluyen el modelo Beta-Bernoulli para recompensas binarias, la distribución Gamma para recompensas positivas asimétricas, la t-*Student* para colas pesadas, la Poisson para datos de conteo, y enfoques empíricos o basados en *bootstrap* para distribuciones desconocidas o multimodales. La elección de la distribución adecuada debe considerar la naturaleza de las recompensas observadas y el desempeño deseado en cada aplicación.

Las complejidades computacionales estimadas para cada algoritmo (Tabla 1) se consolidaron a partir de un análisis de algoritmos, basado en la estructura de sus bucles de decisión y actualización (por ejemplo, ε-*greedy* y UCB tienen operaciones O(1) por paso, mientras que los algoritmos Gradiente y *Thompson Sampling* involucran recorridos O(K)). Este análisis se refinó durante una discusión profunda con un asistente de IA (DeepSeek-V4-Flash, 2026) con fines de claridad y comparación en el contexto de este trabajo.

Agentes	Costo por Paso	1000 steps x 2000 runs
ε-Greedy	O(1) – actualización simple	1 x (línea base)
UCB	O(1) – cálculo simple	~1,2x
Gradiente	O(k) – actualiza todas las acciones	~2-3x (depende de k)
TS(original)	O(k+n) – con np.var() en el historial para un número de accionamientos n de la palanca	~5-10x
TS (optimizado)	O(k) - Welford	~2-3x

Tabla 1 - Comparación de Costos de las Estrategias de Solución

Obs.: La Solución de Welford para *Thompson Sampling* en lugar de usar todas las muestras para calcular la varianza a posteriori, procesa cada muestra de forma incremental.

Como veremos en la sección de Resultados, el *Thompson Sampling* (TS) presenta un desempeño superior a la mayoría de las demás estrategias de solución, tanto en recompensa acumulada como en la tasa de acierto de la acción óptima. Esta ganancia se atribuye a su capacidad de equilibrar exploración y explotación de forma inherentemente adaptativa. Sin embargo, este beneficio viene acompañado de un mayor costo computacional, especialmente cuando se implementa con distribuciones más complejas (como Gamma, t-Student o *bootstrap* (muestreo con reposición de un conjunto de muestras)), que exigen operaciones adicionales por paso. Aun así, en escenarios con un horizonte temporal suficientemente largo o con recursos computacionales adecuados, el TS resulta una elección atractiva y robusta.

Aunque el *Thompson Sampling* presentó el mejor desempeño en términos de recompensa acumulada en la mayoría de los casos, el UCB (*Upper Confidence Bound*) sigue siendo una alternativa ampliamente utilizada en aplicaciones prácticas debido a su mejor relación costo-beneficio. El UCB combina un desempeño cercano al óptimo con un costo computacional significativamente menor, además de una implementación más simple y un ajuste de hiperparámetros menos sensible. Además, el UCB logró superar al TS con la elección de un hiperparámetro c apropiado.

4. **Métricas de Evaluación de las Soluciones:**

La Tabla 2 presenta las métricas utilizadas en la evaluación de los casos de prueba. Esta evaluación se realiza mediante la interpretación de los gráficos generados en cada ejecución.

Para evaluar el desempeño de los algoritmos desde una perspectiva multifacética, los experimentos se analizaron a partir de tres métricas complementarias, cuyas

características, requisitos y aplicabilidades se sintetizan en la **Tabla 2**. Mientras que la recompensa media por paso proporciona un índice de ganancia relativa entre los enfoques, el porcentaje de elección óptima evalúa la precisión estricta del aprendizaje. De manera complementaria, el *Regret* acumulado cuantifica el costo derivado de las decisiones subóptimas tomadas a lo largo del horizonte temporal, permitiendo evaluar las pérdidas de cada política de exploración.

Tipo de métrica	**Recompensa Media por paso en un número de corridas**	**% Elección Óptima**	***Regret* Acumulado**
Mejor adecuación	Análisis comparativo Entre métodos en un mismo entorno	Resultado absoluto/ Practicidad inmediata	Investigación/Ajuste Fino de algoritmos/ Optimización
Conocer μ* (requiere modelado/identificación a priori del entorno)	No necesario	Sí	Sí
Comparación entre estrategias de solución para un mismo entorno	Sí	Sí	Sí
Comparación de la misma estrategia de solución en entornos diferentes	No	Sí	Sí
Muestra el costo del error	Sí, inversamente	No	Sí
Uso	El método A resulta en mayor recompensa media por paso que el método B	90% de las veces acertó la elección óptima	Perdió 500 unidades

Tabla 2 – Métricas de Evaluación de las Soluciones

Obs.: μ*- mayor media estadística de las recompensas entre las k palancas, correspondiente a la elección óptima.

Sea R_t la recompensa obtenida por el agente en el paso t (para $t = 1,2, \dots, n$), donde n es el número total de pasos (horizonte temporal). La **Recompensa Media**

Acumulada $\bar{R}_T$ se define como la media aritmética de todas las recompensas obtenidas hasta el paso T:

$$\bar{R}_T = \frac{1}{n}\sum_{t=1}^{T} R_t \tag{10}$$

El Porcentaje (%) de Elección Óptima puede calcularse mediante la razón entre el número de veces que el bandido óptimo fue elegido N(Q*) y el número de pasos de ejecución del algoritmo (num_steps).

$$\%_\text{Escolha_ó}tima = \frac{N(Q*)}{num_steps} \tag{11}$$

El **Arrepentimiento (regret) Acumulado** tras n pasos se define como la diferencia entre la recompensa total que se obtendría al seleccionar siempre la acción óptima (con media μ*) y la recompensa total efectivamente obtenida por el agente:

$$\boldsymbol{Regret(n) = n \cdot \mu_* - \sum_{t=1}^{n} R_t} \tag{12}$$

donde:

- μ* es la recompensa media de la mejor acción (desconocida por el agente),
- R_t es la recompensa obtenida en el paso t.

Esta métrica es ampliamente utilizada en la literatura [Lattimore & Szepesvári, 2020] por capturar el costo de la exploración de forma más precisa que la recompensa bruta, aunque su cálculo es inviable en el mundo real, en el que no conocemos μ*.

5. **Arquitectura:**

La Figura 4 presenta el Diagrama de Clases utilizado para la ejecución de los casos de prueba de este artículo mediante la implementación de los entornos de los K-Bandidos (Estacionario y No Estacionario) y de los Agentes para diferentes Estrategias de solución (*Alpha – Greedy* y *ε-Greedy*, UCB, *Gradient* y *Thompson Sampling* – TS - optimizado).

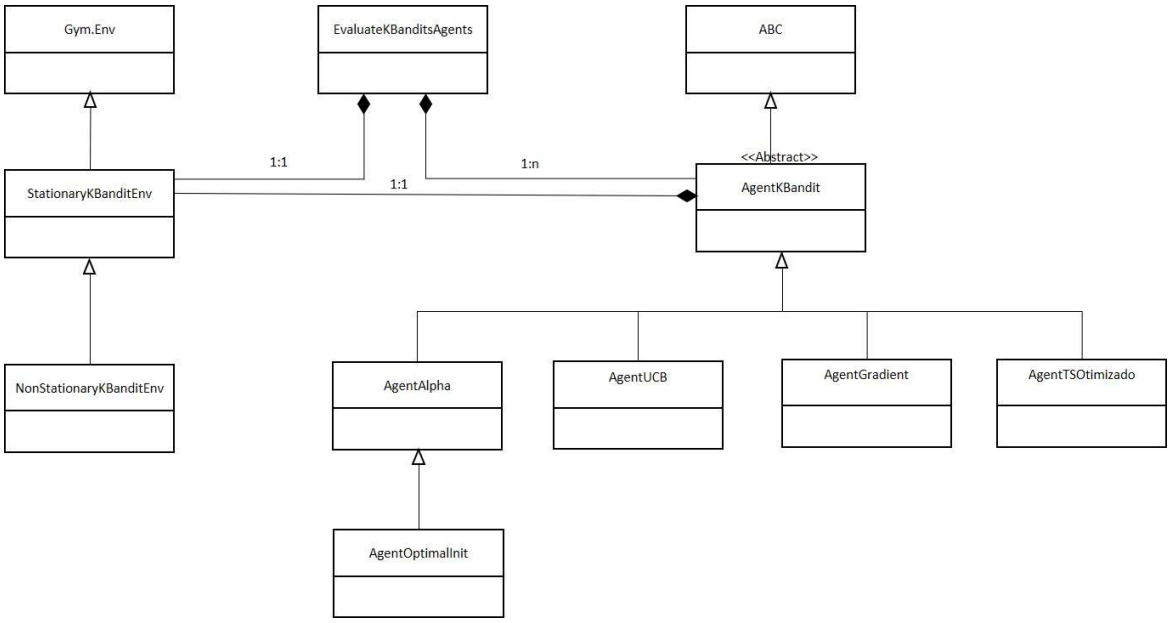


Figura 4-Diagrama de Clases para la Implementación de los K Bandidos

El patrón de diseño *Strategy* [8] confiere flexibilidad a la arquitectura al desacoplar el agente de control y el entorno. Este enfoque facilita la ejecución de múltiples escenarios de prueba y elimina la necesidad de estructuras condicionales complejas en el código.

La aplicación del patrón Template Method [8] permite definir una clase abstracta base para los agentes k-Bandidos, delegando el comportamiento específico a las subclases especializadas de cada estrategia. Esta estructura conserva una interfaz de llamada unificada, simplificando la alternancia entre diferentes agentes de control.

6. Resultados:

La Figura 5 ilustra la tasa de recompensa media y el porcentaje de elección de la acción óptima para los agentes ϵ-greedy ($\epsilon \in \{0; 0{,}01; 0{,}1\}$ con tamaño de paso $\alpha = 1/n$) y para el agente *Thompson Sampling* (TS) optimizado mediante el método de Welford. La simulación se realizó en un entorno estacionario a lo largo de 1.000 pasos y 2.000 repeticiones (*runs*). Se observa que la dinámica de los agentes ϵ-greedy corrobora los resultados de referencia [1, Fig. 2.2], mientras que el agente TS optimizado superó el desempeño de todos los demás métodos evaluados.

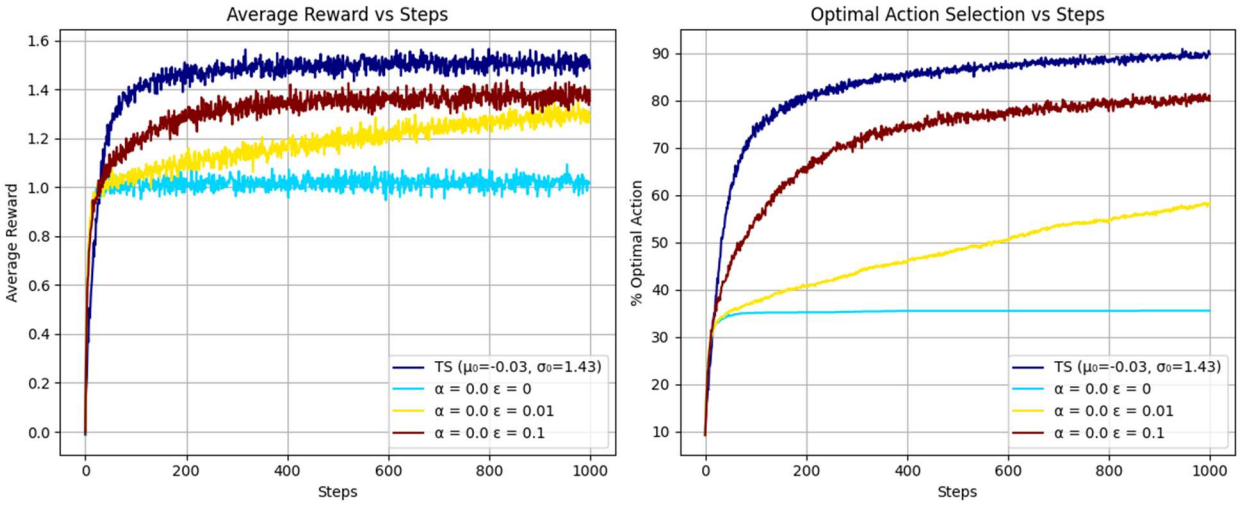


Figura 5- Resultados ε-greedy y TS - Entorno Estacionario

La Figura 6 ilustra la tasa de recompensa media y el porcentaje de elección de la acción óptima para los agentes ϵ-greedy sometidos a diferentes inicializaciones — con valores iniciales optimistas ($Q_1 = +5, \epsilon = 0$) y realistas ($Q_1 = 0, \epsilon = 0{,}1$), ambos con tamaño de paso constante $\alpha = 0{,}1$ — en comparación con el agente Thompson Sampling (TS) optimizado por el método de Welford. La simulación se realizó en un entorno estacionario a lo largo de 1.000 pasos y 2.000 repeticiones (*runs*). Se observa que la dinámica de los agentes ϵ-greedy corrobora los resultados descritos en [1, Fig. 2.3], mientras que el agente TS optimizado superó el desempeño de todos los demás métodos evaluados.

.

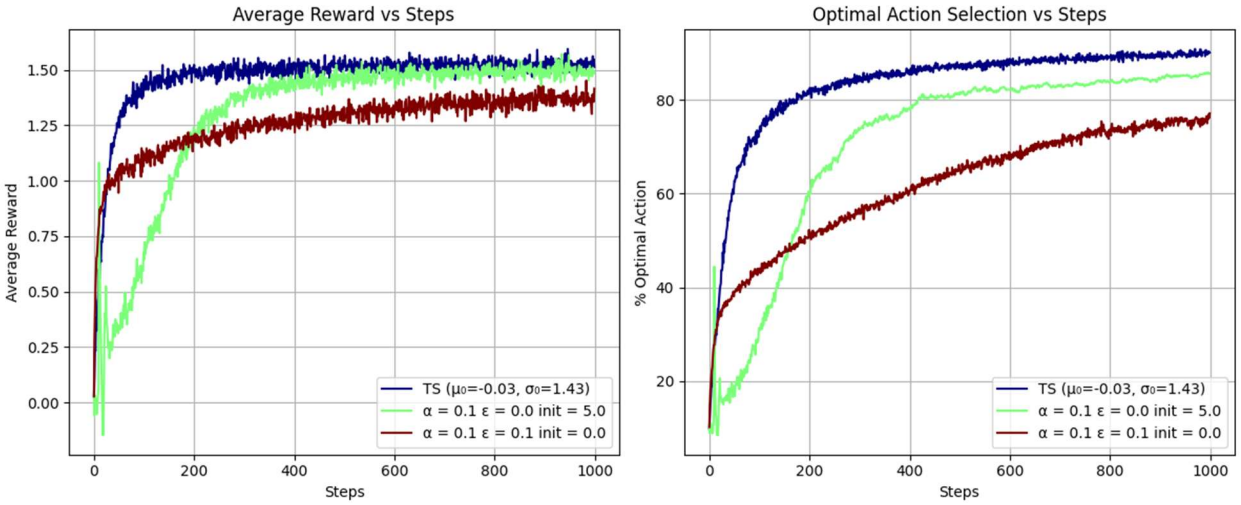


Figura 7 - Resultados ε-greedy con diferentes valores iniciales y TS - Entorno Estacionario

La Figura 7 ilustra la tasa de recompensa media y el porcentaje de elección de la acción óptima para el agente ϵ-greedy ($\epsilon = 0{,}1$ con tamaño de paso $\alpha = 1/n$), para el agente UCB con constante de exploración $c = 2$ y para el agente Thompson Sampling (TS) optimizado por el método de Welford. La simulación se realizó en un entorno estacionario a lo largo de 1.000 pasos y 2.000 repeticiones (*runs*). Se observa que el comportamiento comparativo entre los agentes ϵ-greedy y UCB corrobora los resultados de referencia [1, Fig. 2.4], mientras que el agente TS optimizado superó el desempeño de todos los demás métodos evaluados.

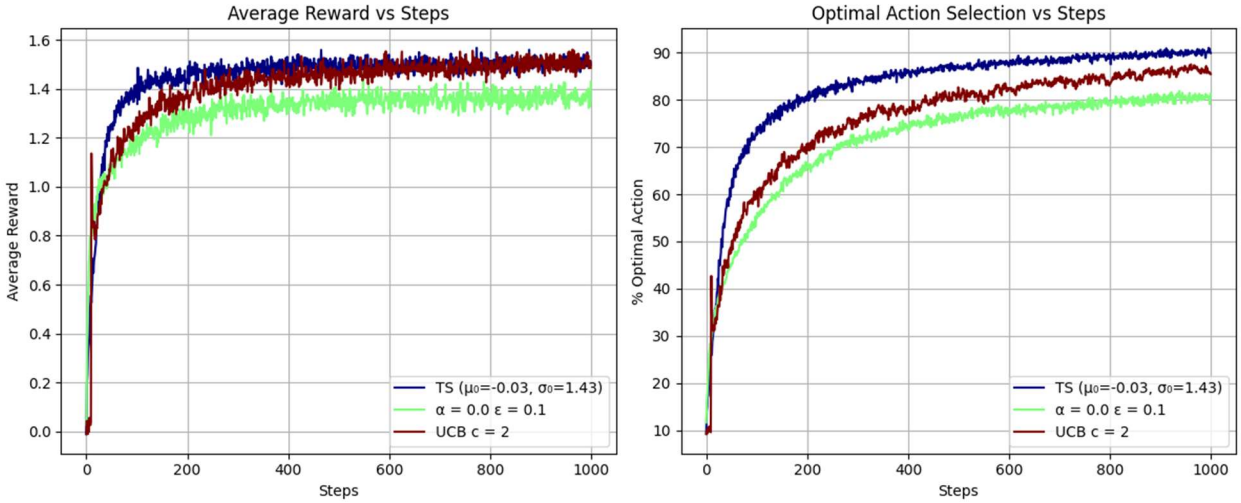


Figura 6- Resultados ε-greedy, UCB y TS - Entorno Estacionario

La Figura 8 ilustra la tasa de recompensa media y el porcentaje de elección de la acción óptima para los agentes de Gradiente con y sin baseline (variando la Learning Rate $lr \in \{0{,}1; 0{,}4\}$, en comparación con el agente Thompson Sampling (TS) optimizado por el método de Welford. La simulación se realizó en un entorno estacionario a lo largo de 1.000 pasos y 2.000 repeticiones (*runs*). Se observa que el comportamiento comparativo entre los agentes de Gradiente corrobora los resultados de referencia [1, Fig. 2.5], evidenciando el papel crucial de la baseline en el desempeño, mientras que el agente TS optimizado superó el desempeño de todos los demás métodos evaluados.

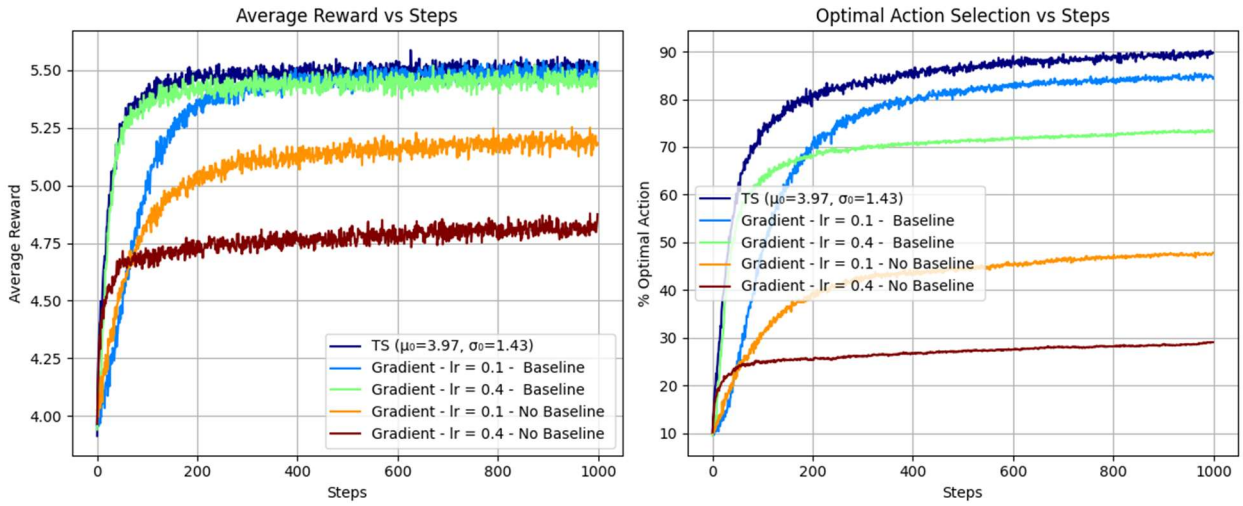


Figura 8 - Resultados Gradiente y TS - Entorno Estacionario

La Figura 9 presenta los mismos resultados obtenidos por Sutton & Barto [1, Fig. 2.6] para 1.000 pasos y 2.000 repeticiones (*runs*), variando los hiperparámetros en un rango de valores para los diversos agentes (con excepción del *Thompson Sampling*), lo que valida la aplicación de estas estrategias en un entorno estacionario. El algoritmo UCB obtuvo el mejor desempeño, alcanzando más del 80% de elecciones óptimas para $c = 1$.

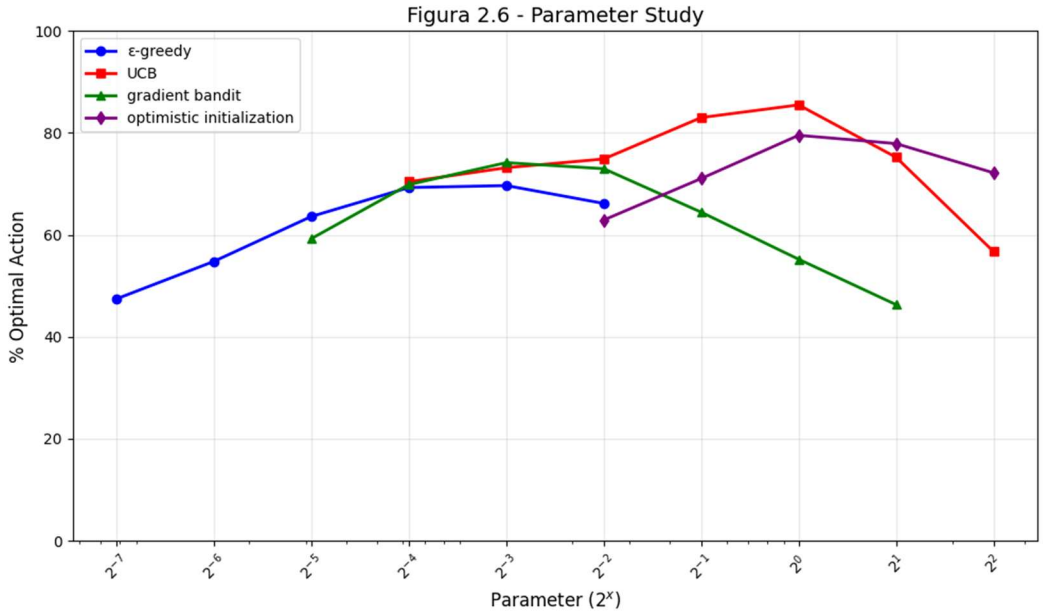


Figura 9- Variación de Hiperparámetros - Entorno Estacionario

La Figura 10 presenta la comparación entre el agente UCB ($c = 1$), el agente ε-greedy ($\varepsilon = 0{,}1$) y el agente *Thompson Sampling* para 1.000 pasos y 2.000 repeticiones (*runs*) en un entorno estacionario. En este escenario, el agente UCB superó al *Thompson Sampling* en desempeño.

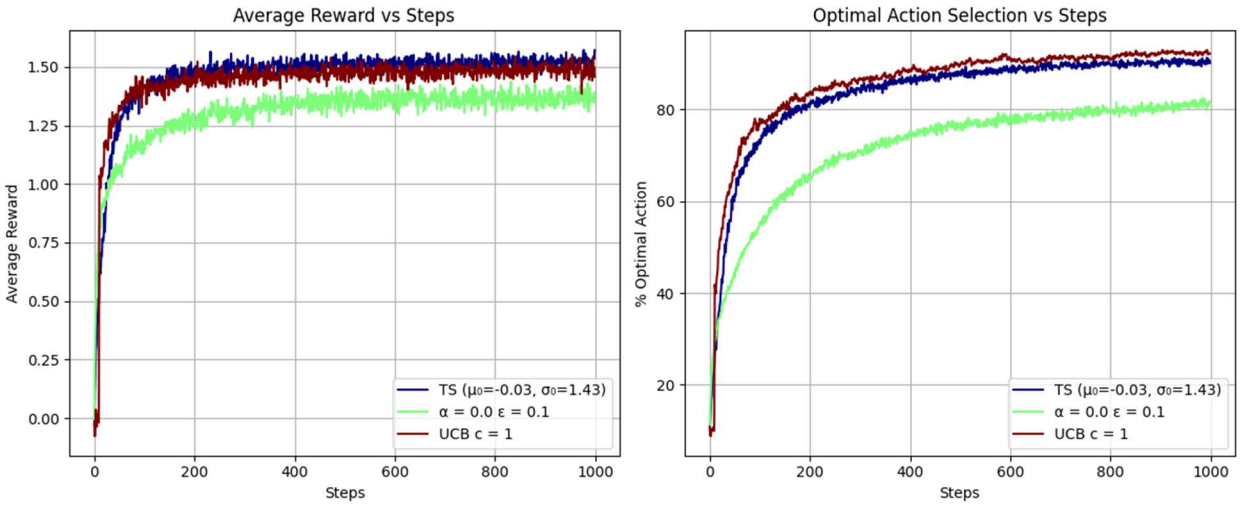


Figura 10- UCB con c = 1 y TS - Entorno Estacionario

Conforme al enunciado del libro [1, Ejercicio 2.11], se obtuvieron los resultados para todos los agentes — con excepción del *Thompson Sampling* — variando los hiperparámetros para el escenario de entorno no estacionario (Figura 11). El desempeño se deterioró en todos los agentes probados. El algoritmo UCB alcanzó alrededor del 51% de elecciones óptimas para $c = 1/8$.

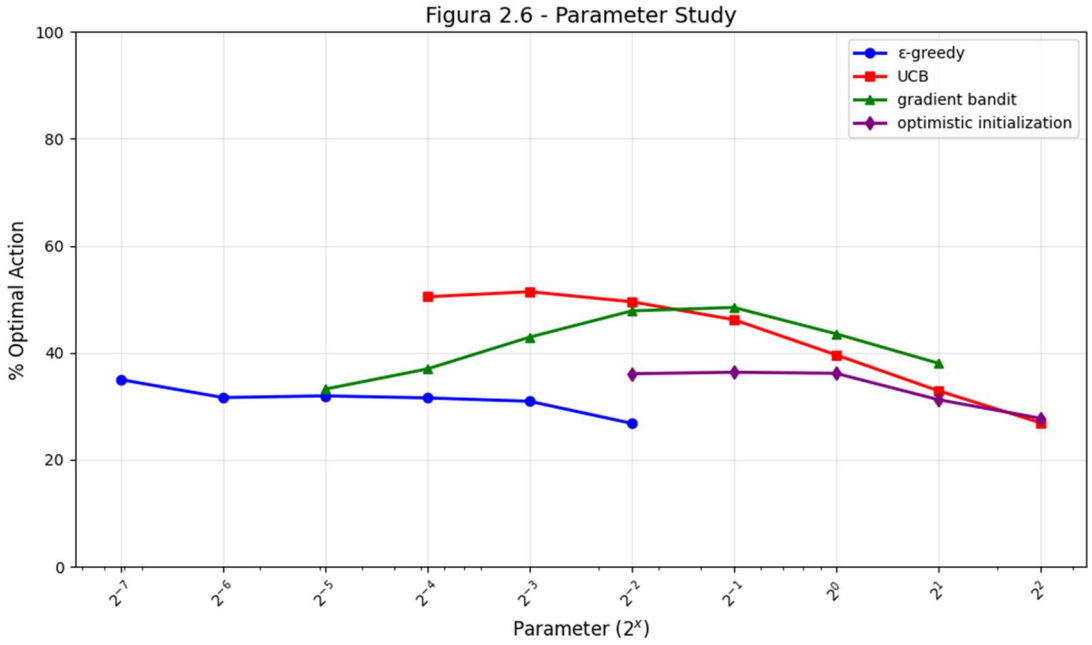


Figura 11- Variación de Parámetros – Entorno No Estacionario

La Figura 12 presenta la comparación entre el agente UCB ($c = 1/8$), el agente ε-greedy ($\varepsilon = 0{,}1$) y el agente *Thompson Sampling* para 1.000 pasos y 2.000 repeticiones (*runs*) en un entorno no estacionario. En este escenario, el agente UCB superó al *Thompson Sampling* en desempeño.

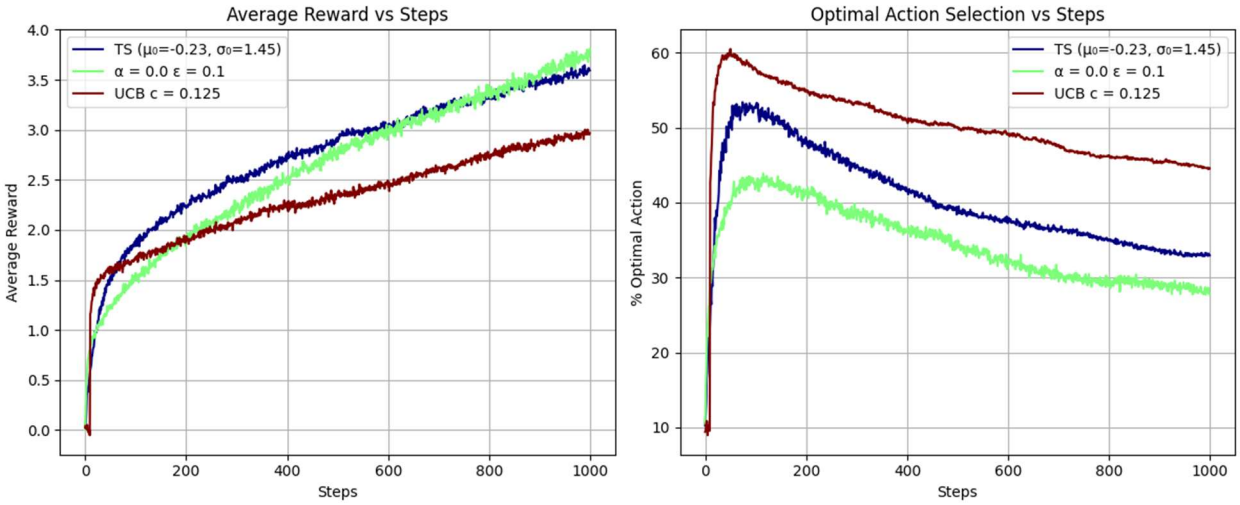


Figura 13- UCB con c = 1 y TS – Entorno No Estacionario

La Figura 13 presenta el caso de prueba de variación de $\alpha \in \{0{,}01; 0{,}1;\ 0{,}5\}$ con ε fijo e igual a 0,1 para el entorno no estacionario. Se observa que la estrategia de utilizar un valor mayor de α presenta mejor desempeño en los pasos iniciales; sin embargo, el resultado se deteriora a medida que el efecto de la no estacionariedad del entorno se acumula con el aumento del número de pasos.

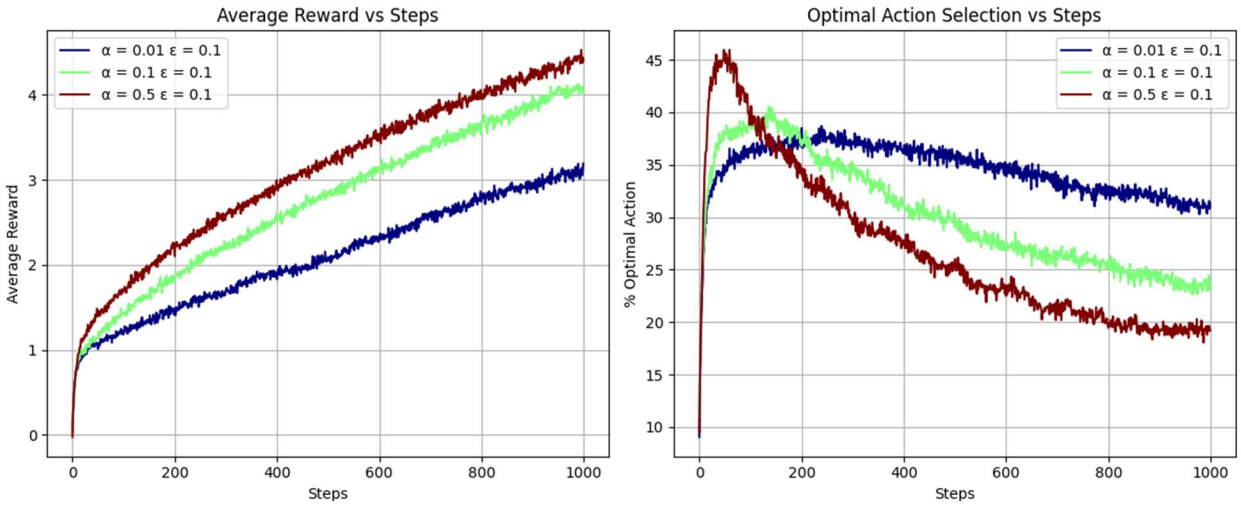


Figura 12- Efecto de la variación de α -Entorno No Estacionario

7. **Conclusión:**

La solución desarrollada en Python fue validada mediante la comparación con los resultados del libro de Sutton & Barto [1] y puede utilizarse de manera flexible para evaluar el desempeño de los agentes estudiados, tanto en entornos estacionarios como no estacionarios, en diversas otras configuraciones.

El algoritmo UCB demostró ser una solución de excelente relación costo-beneficio para el escenario de entorno estacionario.

Todos los agentes probados presentaron un bajo desempeño en entornos no estacionarios, lo que indica la necesidad de emplear técnicas específicas para este tipo de escenario, tales como *Sliding-Window UCB*, *Discounted UCB* y agentes con memoria adaptativa. La referencia [2] presenta una descripción teórica más profunda de diversas técnicas de solución para el problema de los K-Bandidos que no fueron abordadas en el presente artículo.

Queda registrado el agradecimiento a Google por proporcionar el entorno Google Colab con acceso a GPU para la edición, pruebas y análisis de código a través de Gemini integrado, así como a las IAs Gemini y DeepSeek por sus valiosas contribuciones en la revisión del texto, la definición de la arquitectura del *software*, la elaboración y corrección del código en Python y la discusión de los resultados. Se agradece también a la IA Claude por la indicación de referencias bibliográficas sobre aplicaciones reales del problema de los K-Armados Bandidos (*Multi-Armed Bandits*).

.

Referencias:

[1] [Sutton & Barto] *Reinforcement Learning – Second Edition* – The MIT Press - Capítulo 2 - 2018

[2] [Lattimore & Szepesvári] *Bandit Algorithms* – Free Edition – Cambridge Press - 2020

[3] One-armed Bandit, Wiktionary, https://en.wiktionary.org/wiki/one-armed_bandit, acceso el 29/04/2026.

[4] [Bouneffouf & Rish] *A Survey on Practical Applications of Multi-Armed and Contextual Bandits* 02/04/2019

[5] *Artwork Personalization at Netflix, Netflix Blog*, https://netflixtechblog.com/artwork-personalization-c589f074ad76, 07/12/2017, acceso el 30/04/26

[6] [Amrallah, Mohamed, Tran & Sakagushi], MDPI, *UAV Trajectory Optimization in a Post-Disaster Area Using Dual-Energy-Aware Bandits*

[7] [Lapam] *Deep Reinforcement Learning Hands-On -Third Edition* – Capítulo 2 - 2024

[8] [Gamma, Helm, Johnson, Vlissides] *Design Patterns, Elements of Reusable Object-Oriented Software* – Páginas 315 y 325 – 1994

[9] GitHub – https://github.com/kwahcarioca/Reinforcement-Learning/tree/main/K-bandidos/notebooks